

CSS Onboarding — pełna dokumentacja ścieżek rejestracji

Od `css.sembot.com` do realnego onboardingu w Sembocie — krok po kroku, z prawdziwymi zrzutami ekranu, treścią maili i osią czasu. Dla developerów mergujących tę funkcję oraz dla zespołu biznes/CS.

Branch: `codegen/jarek-sciezkaczcssnowysembot-2026-06-23-1333` — `proxy_sembot` + `sembot-laravel`

Wygenerowano: sesja 2026-07-07, odświeżano 2026-07-08 (P8), 2026-07-09 (zrzuty + copy), 2026-07-10 (native-speaker pass + warstwa biznesowa + maile + oś czasu + P11 — atrybucja source + event PostHog), 2026-07-13 (fix propagacji source przez `/auth/css`, korekta domeny wejścia, wdrożenie i weryfikacja linków na `css.sembot.com`) i 2026-07-14 (cofnięcie P8 — trial agencji znów automatyczny przy rejestracji; fix deterministycznego fallbacku `?lang=`; Q&A kontraktu `lang/source` z zespołem `css.sembot.com`)

Spis treści

Kontekst produktowy

Oś czasu — kiedy co się dzieje

Analityka i atrybucja (P11)

Rozdział A — Ścieżka CSS Shop

✓ Wybór profilu — GET /auth/css

Rejestracja — GET/POST /auth/register, POST /signup

Formularz sklepu — GET/POST /css/shop

Strona sukcesu — GET /css/shop/done

✓ CTA "Start Sembot onboarding" → wizar audytu (naprawione w tej sesji)

Rozdział B — Ścieżka CSS Agency

Wybór profilu — GET /auth/css (ten sam ekran co Shop krok 1)

Rejestracja — identyczny formularz, css_type=css-agency

✓ Strona agencji — GET /css/agency (trial znów automatyczny — P8 cofnięte 2026-07-14)

Po kliknięciu mailto — potwierdzenie + zmiana CTA

Wizard Sembota, krok 1 — GET /onboarding/start → /onboarding/domain

Bridge — tworzenie projektu w tle

Cel osiągnięty — realny panel Sembota (Marketer AI + audyt)

Rozdział C — Treści maili

CssShopWelcomeMail — powitanie po CSS Shop

CssShopNotifyMail — zgłoszenie do zespołu CS

CssAgencyWelcomeMail — powitanie agencji + bonus do odebrania

Tabela plików kluczowych

Status MVP i otwarte pytania

Kontekst produktowy

CSS = Google Comparison Shopping Service. Dwa osobne produkty za tą samą rejestracją:

Produkt	Co to jest	Model
Sembot CSS	Darmowa integracja Google Shopping	Darmowy na zawsze dla sklepów; 1 rok gratis dla agencji (White Label), potem €100/mo per market. Niezależny od planu Sembota.
Sembot	Płatna platforma do zarządzania kampaniami	Google/Meta/Microsoft Ads, feed produktowy, automatyzacja, AI agent. 14 dni pełnego dostępu jako bonus za rejestrację przez CSS.

Mechanizm lejka (dlaczego to wszystko istnieje): CSS jest lead magnetem. User rejestruje się po darmowy CSS → dostaje 14-dniowy trial Sembota → ma szansę zobaczyć wartość narzędzia → konwertuje na płatny plan. Każda decyzja opisana w tym dokumencie (segmentacja Shop/Agency, timing aktywacji trialu, treść maili) wynika z tego jednego mechanizmu — CSS sam w sobie nie generuje przychodu, jest przynętą.

Punkt wejścia: `https://css.sembot.com` → `https://new.sembot.com/auth/css` → `https://new.sembot.com/auth/register?type=css-shop` lub `?type=css-agency`. Dwie ścieżki rejestracji różnicowane przez ukryte pole `css_type`, zapisywane na koncie jako `account_type` (enum `AccountType::CSS_SHOP / CSS_AGENCY`).

Korekta domeny (sesja 2026-07-13): wcześniejsza wersja tego dokumentu (i pierwotna wersja specyfikacji przekazanej do zespołu `css.sembot.com`) błędnie wskazywała `app.sembot.com` jako cel linków rejestracji. `app.sembot.com` to osobny, nadal działający, legacy frontend Angular (`sembot-angular`) — nie ma tam ścieżki CSS w ogóle. Cała ścieżka opisana w tym dokumencie (a więc też wszystkie linki z `css.sembot.com`) musi wskazywać na `new.sembot.com` — potwierdzone m.in. przez `signup.go` (rejestracja zawsze wysyła do Laravela Front: "new", żeby link weryfikacyjny wskazywał na `new.sembot.com`) oraz dedykowany rewrite w `chat_render.go`, który automatycznie podmienia stare linki `app.sembot.com / app.sembot.io` na `new.sembot.com` w treściach generowanych przez AI (bug B3 z wcześniejszej sesji). Błąd poprawiony w `DOC_CssOnboardingFunnel.md` i w tym dokumencie.

Uwaga językowa: UI ścieżki wspiera pl/en. Wszystkie 3 maile transakcyjne (Rozdział C) są **tylko po polsku**, niezależnie od locale usera — brak wariantu EN. Warto to potwierdzić z biznesem czy jest to świadome (rynek DACH/inne kraje UE dostają polski mail).

Dokument obejmuje CAŁY flow — od wyboru profilu, przez rejestrację i formularze specyficzne dla CSS, przez treść maili transakcyjnych, aż po faktyczne wejście do wizarda onboardingu Sembota (`/onboarding/audit`). Wszystkie screenshoty nagrane na świeżo zarejestrowanych kontach testowych, ostatnio w sesji 2026-07-10.

Oś czasu — kiedy co się dzieje

Ten sam moment "T+0" (submit formularza rejestracji) prowadzi do bardzo różnych osi czasu dla Shop i Agency w kwestii aktywacji CSS przez Google — ale od 2026-07-14 trial Sembota startuje identycznie w obu ścieżkach (patrz przypis pod tabelą po historię tej decyzji).

Moment	Ścieżka Shop	Ścieżka Agency
T+0 (submit formularza / rejestracja)	Projekt + feed tworzone automatycznie. Mail do CS (e2) i welcome mail do usera (e1) wysyłane od razu. Trial Sembota aktywuje się natychmiast.	Brak automatycznego projektu (dopiero wizard onboardingu Sembota go tworzy). Trial Sembota aktywuje się natychmiast przy rejestracji (od 2026-07-14) — welcome mail (e3) wysyłany od razu, nie czeka na kontakt z CS.
T+0, jeśli user napisze mailem	—	CS dostaje zgłoszenie (mailto, bez potwierdzenia dostarczenia — patrz krok B4) z danymi do konfiguracji White Label.
Aktywacja trialu Sembota	T+0, automatycznie (<code>SubmitShop</code> → <code>CSSStartTrial</code> , plus ogólny mechanizm rejestracji — patrz P8 niżej).	T+0, automatycznie, przy rejestracji (<code>UserRepository::createAccount()</code>) — tak samo jak Shop, od 2026-07-14.
CS aktywuje CSS w Google	T+24-48h.	T+2-4 tygodnie od kompletnych danych.
Koniec 14-dniowego trialu Sembota	T+14 dni — z zapasem ~12 dni nakładania się z aktywnym CSS.	T+14 dni od rejestracji . Świadomie zaakceptowane ryzyko: może wygasnąć zanim White Label CSS w ogóle ruszy (T+14-28 dni) — dokładnie problem, który P8 (2026-07-08) miało rozwiązać. Cofnięte 2026-07-14 na rzecz spójnej, jednolitej polityki produktu.
Dostęp do platformy Sembot	Pełny, od T+0 — niezależnie od statusu trialu (P10: brak egzekwowanego paywalla w kodzie).	Pełny, od T+0 — identycznie jak Shop (P10 dotyczy całej platformy, nie tylko jednej ścieżki).

Historia decyzji (P8): trial dla Agency był automatyczny od początku (2026-06-25) → P8 (2026-07-08) zmienił na ręczną aktywację przez CS, żeby uniknąć ryzyka wygaśnięcia przed startem White Label → cofnięte 2026-07-14 z powrotem na automatyczny, po decyzji biznesowej priorytetyzującej spójność polityki nad tym konkretnym ryzykiem. Pełny kontekst: [proxy_sembot/docs/NOTES_CssOnboardingOpenItems.md](#) , sekcja P8 w [DOC_CssOnboardingFunnel.md](#) .

P10 (finding, nie zadanie): "Dostęp do platformy" w tabeli wyżej jest pełny od razu w OBU ścieżkach, niezależnie od kolumny "Aktywacja trialu Sembota" — bo w kodzie nie ma dziś żadnego middleware faktycznie egzekwującego trial/subskrypcję (sprawdzone: `HasSubscription` i `Paywall` nie są podpięte pod żaden route; nawet gdyby były, realna implementacja `PaywallContract` to no-op). "Aktywacja trialu" jest dziś polem informacyjno-raportowym, nie bramką dostępu.

Analityka i atrybucja (P11)

Kontekst: `css.sembot.com` planuje wersje językowe dla wszystkich krajów UE, ale Sembot realnie wspiera dziś tylko `en / pl`. Niewspierane języki muszą być przekierowywane na ścieżkę `?lang=en` zanim trafią do Sembota — inaczej rejestracja skończy się twardym błędem 422 (`RegisterRequest` waliduje `language` przeciwko enumowi `EN/DE/PL`). Żeby nie stracić informacji "skąd faktycznie przyszedł lead" przy takim wymuszonym przekierowaniu, dodano osobne pole `source`, niezależne od `language`.

Co się zmieniło: `?source=` (np. `css-es`) jest teraz łapane na `/auth/register`, przenoszone przez formularz i zapisywane na `User.source` (Laravel). Event PostHog `user_registered` — odpalany dla KAŻDEJ rejestracji w Semboocie, nie tylko CSS — wzbogacony o `language / account_type` (person property, `$set`) i `source` (atrybucja pierwszego dotknięcia, `$set_once` — nie nadpisywana przez późniejsze eventy).

Scenariusz	Przykładowy event <code>user_registered</code> (<code>\$set / \$set_once</code>)
Zwykła rejestracja (nie-CSS)	<code>language: "pl", account_type: null, source: null</code>
CSS Shop, lead z hiszpańskiej wersji <code>css.sembot.com</code>	<code>language: "en", account_type: "css_shop", source: "css-es"</code>
CSS Agency, lead z niemieckiej wersji	<code>language: "en", account_type: "css_agency", source: "css-de"</code>

Dzięki temu w PostHogu da się odpowiedzieć na pytanie "ile rejestracji przyszło z każdego języka, i czy konwertują tak samo dobrze jak `en/pl`" — bez ręcznego joinowania danych z bazy, bo `source / language / account_type` są trwale zapisane na profilu osoby i dziedziczą je też wszystkie późniejsze eventy tego usera (`subscription_created` itd.).

Fix propagacji przez `/auth/css` (sesja 2026-07-13)

Problem znaleziony przy przygotowaniu linków dla `css.sembot.com`: `ShowCSSPath` (Krok 1, wybór profilu Shop/Agency) renderuje statyczną stronę bez ViewModelu i w ogóle nie czyta `r.URL.Query()`. Karty w `css_path.tpl` mają zahardkodowane hrefy (`/auth/register?type=css-shop / css-agency`) — literalne stringi, nie budowane z parametrów requestu. Efekt: lead wchodzący przez `/auth/css?source=css-es` (zamiast bezpośrednio przez `/auth/register`) tracił atrybucję w momencie kliknięcia karty, bo `source` nigdzie nie było przenoszone dalej.

Rozwiązanie — `middleware.WithSource` (`internal/middleware/source.go`), wzorzec analogiczny do już istniejącego `WithLocale / ?lang=`:

- Middleware wpięty globalnie w chain (`cmd/server/main.go`) — działa na KAŻDEJ stronie, nie tylko `/auth/register`.
- Łapie `?source=` na **pierwszym** trafieniu w dowolną stronę funnela i zapisuje w cookie `sembot_source` (30 dni) — analogicznie jak `WithLocale` robi to z `sembot_locale` dla `?lang=`.
- Różnica względem `lang`:** cookie wygrywa nad query (first-touch attribution) — w przeciwieństwie do `lang`, gdzie user może świadomie przełączyć język w każdej chwili, `source`

ma sens tylko jako atrybucja PIERWSZEGO dotknięcia leada; przypadkowa nawigacja bez tagu (albo z innym) nie powinna nadpisać już zapisanej wartości.

- ShowSignup / DoSignup czytają teraz przez `middleware.Source(r)` zamiast tylko z query/form — działa niezależnie czy user trafia na `/auth/register` bezpośrednio, czy przez `/auth/css`.
- `sanitizeSource` przeniesiony z `internal/handlers/signup.go` do `internal/middleware/source.go` jako `SanitizeSource` (eksportowane) — jedno miejsce prawdy zamiast duplikatu, regex bez zmian (`^[a-zA-Z0-9-]{1,64}$`).
- Testy: `TestSanitizeSource` (przeniesiony) + nowy `TestWithSource` (capture z query, first-touch/cookie-wins, brak source, sanityzacja złego inputu) w `internal/middleware/source_test.go`. `go build ./...`, `go vet ./...`, `go test ./...` — zielone.

Efekt: `source` przeżywa oba warianty linkowania z `css.sembot.com` — czy to wspólny wybierak profilu (`/auth/css`), czy bezpośredni link do konkretnej ścieżki (`/auth/register?type=...`). Marketing nie musi wiedzieć o tym rozróżnieniu przy budowaniu linków.

Wdrożenie po stronie `css.sembot.com` (sesja 2026-07-13, zweryfikowane)

Finalna lista linków przekazana i wdrożona przez zespół odpowiedzialny za `css.sembot.com`:

Wersja	Element	URL
PL	nav-CTA (topbar, wybór profilu)	<code>new.sembot.com/auth/css?lang=pl&source=css-pl</code>
EN	nav-CTA (topbar, wybór profilu)	<code>new.sembot.com/auth/css?lang=en&source=css-en</code>
PL	CTA "Sklep" (/ecommerce, profil znany)	<code>new.sembot.com/auth/register?type=css-shop&lang=pl&source=css-pl</code>
EN	CTA "Sklep" (/ecommerce, profil znany)	<code>new.sembot.com/auth/register?type=css-shop&lang=en&source=css-en</code>
PL	CTA "Agencja" (/agency, profil znany)	<code>new.sembot.com/auth/register?type=css-agency&lang=pl&source=css-pl</code>
EN	CTA "Agencja" (/agency, profil znany)	<code>new.sembot.com/auth/register?type=css-agency&lang=en&source=css-en</code>

17 linków w 8 plikach szablonów po stronie `css.sembot.com`. Zweryfikowane przez render server-side HTML (curl po wszystkich podstronach PL/EN) — zero linków do `app.sembot.com`, poprawny encoding `&` w atrybutach href. Tenant `css.sembot.com` ma dziś tylko 2 wersje językowe (EN root, PL `/pl`) — poniższy wzorzec dla niewspieranych języków UE nie miał jeszcze zastosowania, zostaje udokumentowany pod przyszłą ekspansję językową.

Wzorzec gotowy pod przyszłe niewspierane języki (es, de, fr, it, nl...) — `<kod_języka>` to dwuliterowy kod ISO wersji strony:

- Jeśli dana wersja nie rozróżnia jeszcze Sklep/Agencja: `new.sembot.com/auth/css?lang=en&source=css-<kod_języka>`
- Jeśli ma już osobne CTA: `new.sembot.com/auth/register?type=css-shop&lang=en&source=css-<kod_języka>` / `type=css-agency` odpowiednio

Oba warianty działają poprawnie dzięki fixowi propagacji opisanemu wyżej — nie trzeba już rozróżniać, przez który punkt wejścia trafia lead.

Decyzja analityczna — dlaczego `source` jest ustawiane też dla PL/EN, mimo że jest tam redundantne: dla wspieranych języków `source=css-pl / css-en` nie niesie żadnej nowej informacji ponad to, co już dają `account_type` (jednoznacznie: to lead z CSS) + natywne `language` (jednoznacznie: wersja językowa) — w przeciwieństwie do niewspieranych języków, gdzie `language` jest wymuszane na `en` i `source` jest jedynym źródłem prawdy o faktycznym pochodzeniu. Mimo tej redundancji zdecydowano zostawić `source` wypełnione wszędzie, bo koszt jest zerowy (mechanizm już istnieje, `$set_once` — zapisuje się raz), a alternatywa (`source` puste dla większości ruchu CSS) tworzy realne ryzyko: analityk filtrujący "leady z CSS" przez `source LIKE 'css-%'` — naturalna interpretacja nazwy pola — cicho zaniżyłby wynik o całą populację PL/EN, bez żadnego błędu czy ostrzeżenia. Ujednolicona populacja pola eliminuje tę klasę pomyłek.

Status:  **zamknięte end-to-end** — kod (middleware + testy), dokumentacja (`DOC_CssOnboardingFunnel.md`) i wdrożenie po stronie `css.sembot.com` (linki live, zweryfikowane) są zgodne. Nie ma już otwartego punktu "do zrobienia po stronie biznesu" dla P11 — jedyne co zostaje otwarte to faktyczne uruchomienie niewspieranych wersji językowych na `css.sembot.com` w przyszłości (wzorzec URL już udokumentowany i gotowy do użycia, patrz tabela wyżej).

Deterministyczny fallback dla `?lang=` (fix, sesja 2026-07-14)

Znalezisko przy okazji Q&A z zespołem `css.sembot.com` (pytanie: "gdy dodamy niewspierany język np. DE, czy mamy sami degradować `lang=de` na `lang=en`, czy Wasz backend to robi?"):
`WithLocale` do tej pory **nie** robił tego deterministycznie. Nieobsługiwany kod w `?lang=` był po prostu ignorowany (`bundle.Has(q)` = false → nic się nie działo z tym parametrem), a middleware spadał do zgadywania języka z nagłówka `Accept-Language` przeglądarki leada. W skrajnym, ale realnym przypadku (np. przeglądarka niemieckiego usera wysyłająca `Accept-Language: de,pl;q=0.5`) mogło to wylądować na `pl` zamiast na domyślnym `en` — czyli ten sam link marketingowy mógł dać różny wynik w zależności od konfiguracji przeglądarki, a nie od parametru w URL-u.

Fix: jawny, nieobsługiwany `?lang=` mapuje się teraz wprost na `bundle.Fallback()` (dziś: `en`) i jest persystowany w cookie — bez dalszego sniffingu `Accept-Language` . Ten sam link zawsze daje ten sam efekt, niezależnie od przeglądarki. Plik: `internal/middleware/i18n.go` . Testy: `internal/middleware/i18n_test.go` (4 przypadki — wspierany `?lang=` , niewspierany → deterministyczny fallback zamiast sniffingu potwierdzony wprost przeciwko myląco skonfigurowanemu `Accept-Language` , brak `?lang=` → normalny sniffing, cookie wygrywa nad kolejnym `?lang=`).

Efekt: zespół `css.sembot.com` może budować JEDEN schemat linków, bez rozgałęzień po swojej stronie — zawsze `lang=<realny_kod_wersji_strony>` (np. `lang=de` , `lang=es` ...), niezależnie czy Sembot już wspiera ten język. Mapowanie na fallback dzieje się w Sembocie, nie po ich stronie.

Q&A z zespołem `css.sembot.com` (2026-07-14) — kontrakt `lang/source` dla integracji

Cztery pytania od zespołu budującego linki wejściowe, z odpowiedziami zweryfikowanymi wprost w kodzie (nie z pamięci):

Pytanie	Odpowiedź
Kto mapuje niewspierany <code>lang=</code> na fallback?	Sembot, deterministycznie (patrz fix wyżej) — <code>css.sembot.com</code> zawsze wysyła realny kod strony, nigdy nie musi sam

	degradować. Wspierane dziś: <code>pl</code> , <code>en</code> .
Czy <code>source</code> ma walidowany format?	Nie — czysty wolny tekst, jedyna reguła to regex <code>^[a-zA-Z0-9-]{1,64}\$</code> (<code>SanitizeSource</code>). Brak rejestru/allowlisty w PostHogu — co wyślą, to 1:1 (case-sensitive) trafia jako wartość pola. Rekomendacja: małe litery, konsekwentnie (<code>css-pl</code> , nie <code>CSS-PL</code>).
Czy <code>lang=</code> przełącza widoczny język formularza, czy tylko atrybut konta?	Przełącza cały widoczny UI — resolved locale idzie przez <code>ui.WithUI(ctx, loc, ...)</code> do każdego <code>ui.T(ctx, key)</code> w szablonach. User z polskiej podstrony zobaczy polski formularz, nie tylko polskie <code>language</code> na koncie.
<code>source / lang</code> — first-touch czy last-touch?	Różnie, świadomie: <code>lang</code> jest last-touch (każdy kolejny <code>?lang=</code> nadpisuje cookie — user może świadomie przełączyć język). <code>source</code> jest first-touch (raz zapisany, nie nadpisywany przez 30 dni) — atrybucja ma wskazywać PIERWSZE dotknięcie leada, nie ostatni klik.

Konsekwencja czwartej odpowiedzi, wprost zakomunikowana zespołowi css.sembot.com:
user, który odwiedzi PL, potem EN, potem zarejestruje się — wyląduje z `language=en` (ostatni klik) ALE `source=css-pl` (pierwszy klik). To zamierzone rozjechanie dla poprawności atrybucji, nie błąd — ale oznacza że przy analizie danych `lang` i `source` nie zawsze opisują tę samą wizytę dla userów przełączających język w trakcie sesji.

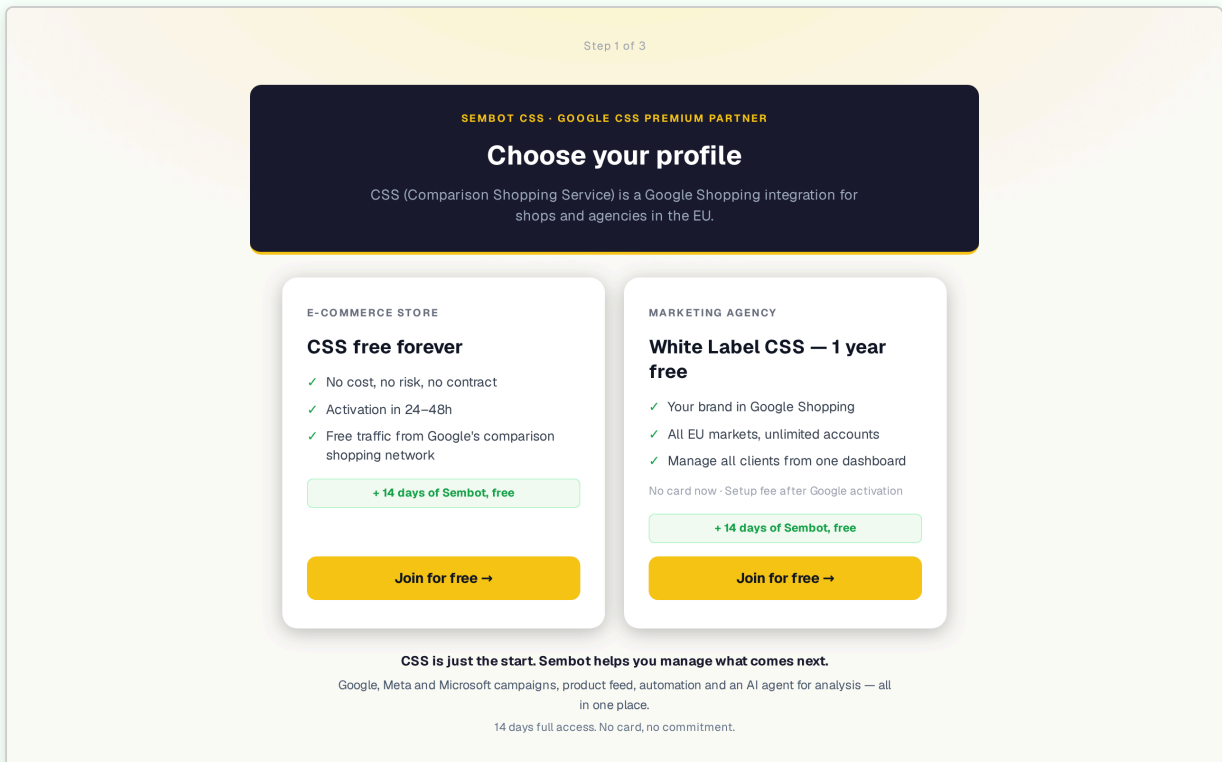
Rozdział A — Ścieżka CSS Shop

auth/css → register?type=css-shop → /css/shop → /css/shop/done → onboarding/audit

1 Wybór profilu — GET /auth/css NAPRAWIONE

Trasa: GET /auth/css

Handler: Auth.ShowCSSPath — internal/handlers/auth.go:108



01-wybor-profilu.png

CO WIDZI UŻYTKOWNIK

Ciemna karta hero (spójna z css.sembot.com) + dwie karty wyboru, wizualnie równorzędne (bez badge'a "POPULAR" — usunięty w P8-audycie, commit [1c9c68b](#)): "E-commerce Store" i "Marketing Agency". Pod kartami blok wyjaśniający czym jest Sembot (Google/Meta/Microsoft, feed, AI agent, 14 dni bez karty).

KONTEKST BIZNESOWY

CSS to darmowy lead magnet — segmentacja Shop/Agency na starcie pozwala dopasować dalej model przychodu do właściwego typu klienta: sklep dostaje CSS za darmo na zawsze, agencja rok gratis a potem €100/mies. za rynek. To inny model monetyzacji, więc komunikat na tej stronie musi jasno rozróżniać oba segmenty — poprzednia wersja subtitle'a obiecywała "free" jednakowo dla obu, co było nieprawdą dla agencji (naprawione, patrz niżej).

CO SIĘ DZIEJE POD MASKĄ

- Handler renderuje statyczną stronę wyboru — brak zapisu do sesji ani wywołań API na tym etapie.
- CTA "Join for free ->" na karcie Sklep prowadzi do `/auth/register?type=css-shop`; karta Agencja do `/auth/register?type=css-agency`.
- Blok wyjaśniający Sembota (`.css-path-sembot-info`) — zaimplementowany jako P3 w `DOC_CssOnboardingFunnel.md`, commit [8971e60](#).
- **Naprawione w sesji 2026-07-09:** tytuł bloku ("CSS is just the start...") miał kolor `#e2e8f0` (prawie biały), dobrany pod martwą regułę `body{background:#1a1a2e}` — w praktyce wrapper `.auth` (`login.css`) zawsze maluje jasne tło, więc tytuł był ledwo widoczny. Zmieniono na `#1a1a2e / #475569`, usunięto martwą regułę. Commit [bc67304](#).
- **Naprawione w sesji 2026-07-09:** subtitle w hero ("CSS ... is free Google Shopping integration for shops and agencies") twierdził że CSS jest "free" jednakowo dla obu segmentów — nieprawda dla agencji (1 rok gratis, potem €100/mo per market, patrz karta Agencji obok). Porównano z definicją na

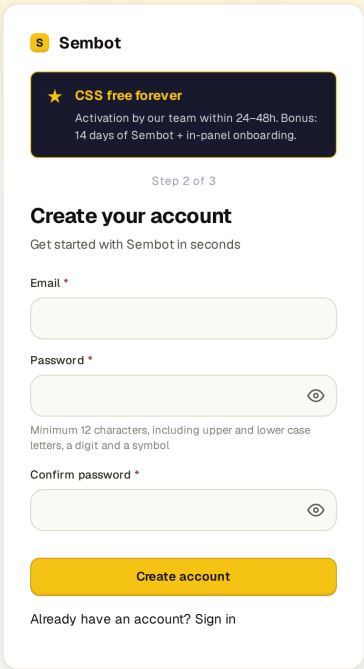
css.sembot.com (neutralna, bez "free" — ceny rozdzielone per segment niżej) i usunięto "free" z subtitle'a. Commit `26615b2`. Rozważano też dodanie badge'a "Google Comparison Shopping Premium Partner" (jak na css.sembot.com) — świadomie odłożone (asset ma "2025" wypalone w grafice, status aktualności do potwierdzenia z zespołem przed użyciem w produkcji).

- **Naprawione w sesji 2026-07-10:** native-speaker pass na angielskim copy całej ścieżki (7 miejsc z "14 days Sembot" → "14 days of Sembot", brakujące przedimki/zaimki, niespójne czasowniki) — commit `f6ac2c3`.
- **Naprawione w sesji 2026-07-13:** ten ekran ma zahardkodowane hrefy kart (`/auth/register?type=css-shop / css-agency`) i nie przepisuje dalej własnego query stringa — link `/auth/css?source=...` sam z siebie gubił atrybucję po kliknięciu karty. Naprawione strukturalnie w middleware, nie w tym templu — patrz sekcja "Analityka i atrybucja" niżej.

2 Rejestracja — GET/POST /auth/register, POST /signup

Trasa: GET /auth/register?type=css-shop → POST /signup

Handler: Auth.ShowSignup (signup.go:25) / Auth.DoSignup (signup.go:50)



02-rejestracja-shop.png

CO WIDZI UŻYTKOWNIK

Ciemny baner nad formularzem: "CSS bezpłatny na zawsze — Aktywacja przez nasz zespół w 24-48h. Bonus: 14 dni Sembota + onboarding w panelu." Standardowy formularz: email + hasło (walidacja inline Alpine: min. 12 znaków, wielka/mala litera, cyfra, symbol) + potwierdzenie hasła.

KONTEKST BIZNESOWY

Rejestracja to jedyny moment, w którym zapisujemy dane decydujące o dalszej monetyzacji konta (`account_type`). Baner nad formularzem musi jasno komunikować że to rejestracja do konkretnego programu CSS, nie generyczne konto Sembot — inaczej user może się zdziwić gdzie trafił i porzucić formularz.

CO SIĘ DZIEJE POD MASKĄ

- `ShowSignup` czyta `?type=` z query, sanityzuje do `css-shop` / `css-agency` / pusty (nieznana wartość = zwykła rejestracja) i przekazuje jako ukryte pole `css_type` w formularzu.
- `DoSignup` (POST /signup, HTMX) waliduje zgodność haseł lokalnie, woła Laravel `POST /api/v1/register` (`AccountType` mapowane z myślnika na podkreślnik po stronie Laravela — `LoginController.php:180`), następnie auto-loguje przez `POST /login` żeby od razu mieć JWT.
- Destynacja po sukcesie zależy od `css_type` : `css-shop` → `/css/shop` , `css-agency` → `/css/agency` , inne → `/onboarding/start` (signup.go:108-113).
- **Odtworzony w tej sesji real problem środowiskowy (nie kod produkcyjny):** w świeżo zaimportowanym devcontainerze auto-login po rejestracji failował z „Personal access client not found” — brakowało Laravel Passport personal access client. Fix: `php artisan passport:client --personal` . Osobno: kolumna `users.front` (śledzi z jakiego frontu poszła rejestracja — wpływa na URL linku weryfikacyjnego) była nieobecna dopóki nie dogoniono migracji landlord po imporcie snapshotu.
- **P11 — dodane w sesji 2026-07-10:** formularz przyjmuje teraz też `?source=` (np. `css-es`) — ukryte pole analogiczne do `css_type` , sanityzowane (`sanitizeSource` , tylko

alfanumeryczne+myślnik, max 64 znaki), przenoszone do `User.source` w Laravelu. Czysto atrybucyjne, niewidoczne w UI — patrz Rozdział "Analityka i atrybucja" niżej.

3 Formularz sklepu — GET/POST /css/shop

Trasa: GET /css/shop → POST /css/shop

Handler: CSSOnboarding.ShowShop (css_onboarding.go:37) / CSSOnboarding.SubmitShop (css_onboarding.go:52)

Step 3 of 3

SEMBOT CSS · GOOGLE CSS PREMIUM PARTNER

Your CSS is free forever

Fill in the details — our team will activate your CSS within 24–48h.

★ **BONUS: 14 days of Sembot + in-panel onboarding**

Store name *

Testowy Sklep PDF

Product feed URL *

https://testowy-sklep-pdf.example.com/feed.xml

Product feed URL from your store. Shopify: Sales Channels → Google → Feed URL. WooCommerce: Product Feed plugin. PrestaShop: Modules → Google Merchant Center.

Store domain *

testowy-sklep-pdf.example.com

GMC ID (Google Merchant Center) (optional)

123456789

Where to find your GMC ID? Log in to [merchants.google.com](#) → number in the top-left or in the URL (?a=XXXXXXXX). No account? [Create account](#) — takes about 10 minutes.

04b-formularz-wypelniony.png

CO WIDZI UŻYTKOWNIK

Pola: nazwa sklepu (wymagane), URL feeda produktowego (wymagane, z podpowiedzią Shopify/WooCommerce/PrestaShop), domena sklepu (wymagane, auto-fill z hosta feed URL przez Alpine `x-on:input`), GMC ID (opcjonalne, domyślnie zwinięte za linkiem-togglę, z instrukcją gdzie znaleźć w Merchant Center). CTA: "Activate CSS and go to your panel".

KONTEKST BIZNESOWY

Minimalny formularz (3 pola wymagane) obniża barierę wejścia na etapie, na którym user jeszcze nie zaangażował się w produkt. GMC ID jest domyślnie zwinięte, bo część sklepów jeszcze go nie ma na starcie — wymuszanie go od razu realnie blokowałoby konwersję (fix UX z audytu, sesja 2026-07-07).

CO SIĘ DZIEJE POD MASKĄ

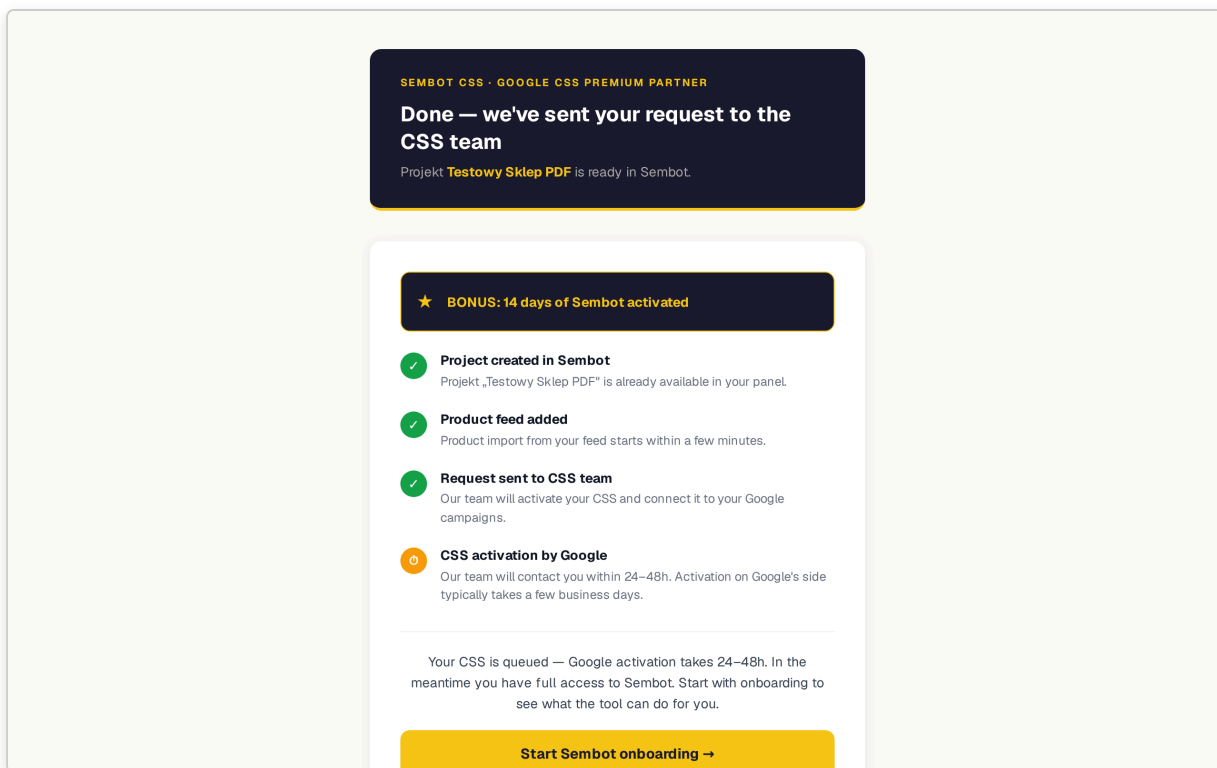
- `SubmitShop` woła sekwencyjnie: `CSSCreateProject` (POST `/api/v1/css/shop/project` — auto-weryfikuje email usera server-side, tworzy projekt typu `SIMPLE`), `CreateInputFeed` (feed z domyślnym mapowaniem 18 pól Google Shopping, `api.DefaultCSSFeedMapping()`), `CSSShopNotify` (POST `/api/v1/css/shop/notify` — wysła DWA maile naraz, patrz Rozdział C), `CSSStartTrial` (POST `/api/v1/css/start-trial` — aktywuje 14-dniowy trial natychmiast).
- Błędy `CreateInputFeed`, `CSSShopNotify` i `CSSStartTrial` są nieblokujące (tylko `log.Printf`) — projekt i przekierowanie na success page następują nawet gdy feed/notify/trial zawiodą. Tylko błąd `CSSCreateProject` zatrzymuje flow i re-renderuje formularz z komunikatem błędu.
- Real bug odtworzony w tej sesji:** w devcontainerze z pustą tabelą `tenants` (landlord) i `DB_PROJECTS_PER_TENANT>1`, `CSSCreateProject` zwraca 500 (`StoreProject.php:89` — `Tenant::orderBy('id','desc')->first()` zwraca `null`, kod nie ma gałęzi na „zero tenantów”).

Środowiskowe, nie blokuje mergowania tego PR-a, ale warto wiedzieć że pierwsza rejestracja CSS Shop w świeżym devcontainerze bez zaimportowanego snapshotu z realnymi tenantami zawiedzie.

4 Strona sukcesu — GET /css/shop/done

Trasa: GET /css/shop/done?ws=&proj=&name=

Handler: CSSOnboarding.ShowShopDone (css_onboarding.go:142)



05-potwierdzenie-shop.png

CO WIDZI UŻYTKOWNIK

Checklistka: ✓ Projekt stworzony, ✓ Feed dodany, ✓ Zgłoszenie do CS, ⌚ Aktywacja CSS przez Google (24–48h). Bonus box "BONUS: 14 days of Sembot activated". CTA primary: "Start Sembot onboarding →", secondary: link do panelu projektu.

KONTEKST BIZNESOWY

To jest właściwy cel całego lejka, nie CSS samo w sobie: user dostał darmowy CSS, ale ta strona pokazuje mu że dostaje też 14 dni Sembota — płatnej platformy. Ten moment decyduje, czy user w ogóle spróbuje Sembota, czy tylko odbierze CSS i zniknie. "Aktywowane" jest tu prawdziwe (w przeciwieństwie do wcześniejszej wersji dla Agencji, patrz Rozdział B krok 3) — aktywacja CSS Shop trwa 24-48h, czyli wygodnie mieści się w 14-dniowym trialu.

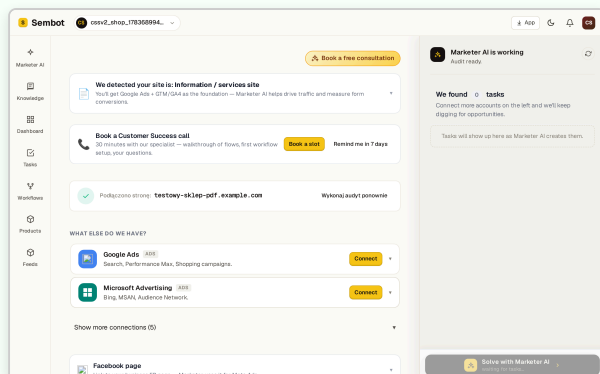
CO SIĘ DZIEJE POD MASKĄ

- Parametry `ws` / `proj` / `name` idą przez query string (nie ma w nich wrażliwych danych — komentarz w kodzie to uzasadnia); sesja jest i tak weryfikowana przez `requireSession`.
- Primary CTA linkuje na `/p/{ws}/{proj}/onboarding/audit` — scoped route, bezpośrednio do wizarda audytu Sembota (P1 z `DOC_CssOnboardingFunnel.md`, commit `4797371`).
- `CssShopWelcomeMail` (osobny mail DO USERA, nie do CS) jest wysyłany z `CssOnboardingController::shopNotify()` gdy request zawiera `project_url` — pełna treść w Rozdziale C.

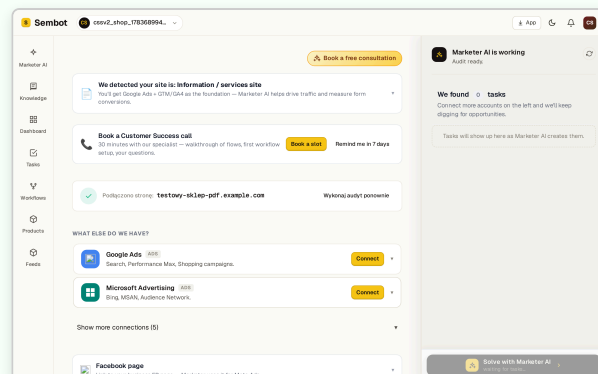
5 CTA "Start Sembot onboarding" → wizard audytu (naprawione w tej sesji) **NAPRAWIONE**

Trasa: GET /p/{ws}/{proj}/onboarding/audit (kliknięcie CTA z kroku 4)

Handler: Onboarding.ShowAudit (onboarding.go:2008)



PRZED - 09-onboarding-audit-shop-populated.png



PO - 08-onboarding-audit-shop.png

CO WIDZI UŻYTKOWNIK

Po kliknięciu CTA użytkownik widzi pełny panel Sembota (Marketer AI, lista integracji do podłączenia, wynik audytu strony) — patrz zrzut "PO" obok.

KONTEKST BIZNESOWY

Ten bug realnie niszczył konwersję CSS→Sembot: user klikał dokładnie ten CTA, który miał go wciągnąć w płatny produkt, i lądował na pustej liście zadań. Najbardziej prawdopodobny moment odbicia się od produktu tuż po tym jak dostał to, po co przyszedł (darmowy CSS) — część userów sprzed fixu prawdopodobnie uznała że produkt jest pusty/nie działa i się nie wróciła.

CO SIĘ DZIEJE POD MASKĄ

- **Bug znaleziony i odtworzony w tej sesji:** kliknięcie "Start Sembot onboarding →" na stronie sukcesu CSS Shop (krok 4) przenosiło na /p/{ws}/{proj}/tasks (pusta lista "No results", zrzut "PRZED") zamiast na wizard audytu, mimo że href linku był poprawny (/p/{ws}/{proj}/onboarding/audit) i strona docelowa istniała. Odtworzone dla dwóch niezależnych świeżych kont (ws=3920/proj=1438 oraz ws=3930/proj=1448).
- **Root cause:** SubmitShop (css_onboarding.go) tworzył projekt przez CSSCreateProject , ale nigdy nie utrzymywał scope'u nowego projektu (ProjectSlug , OnboardingStep) w sesji. Kliknięcie CTA trafiało w middleware.OnboardingRedirect (OnboardingComplete()==false dla świeżego CSS-usera) → /onboarding/start → Onboarding.ShowDomain → firstProjectDest , który przez fallback ListProjects znajdował właśnie stworzony projekt CSS i uznawał usera za "istniejącego z projektem" → 303 na /tasks .
- **Fix:** dodano pole Session *session.Manager do handlera CSSOnboarding (wpięte w cmd/server/main.go) i, zaraz po udanym CSSCreateProject w SubmitShop , utrwalono sess.WorkspaceSlug , sess.ProjectSlug i sess.OnboardingStep = stepAudit przez h.Session.Set(w, sess) — dokładnie ten sam wzorzec, którego już używa Onboarding.SaveDomain w standardowym wizardzie. Commit: 883b60a .
- Dodano test regresyjny w css_onboarding_test.go (TestCSSOnboardingSubmitShop_HappyPath) sprawdzający że sesja po submisie ma poprawny ProjectSlug / OnboardingStep . Zweryfikowane end-to-end na świeżym koncie (Playwright) — zrzut "PO" pochodzi z tej weryfikacji.

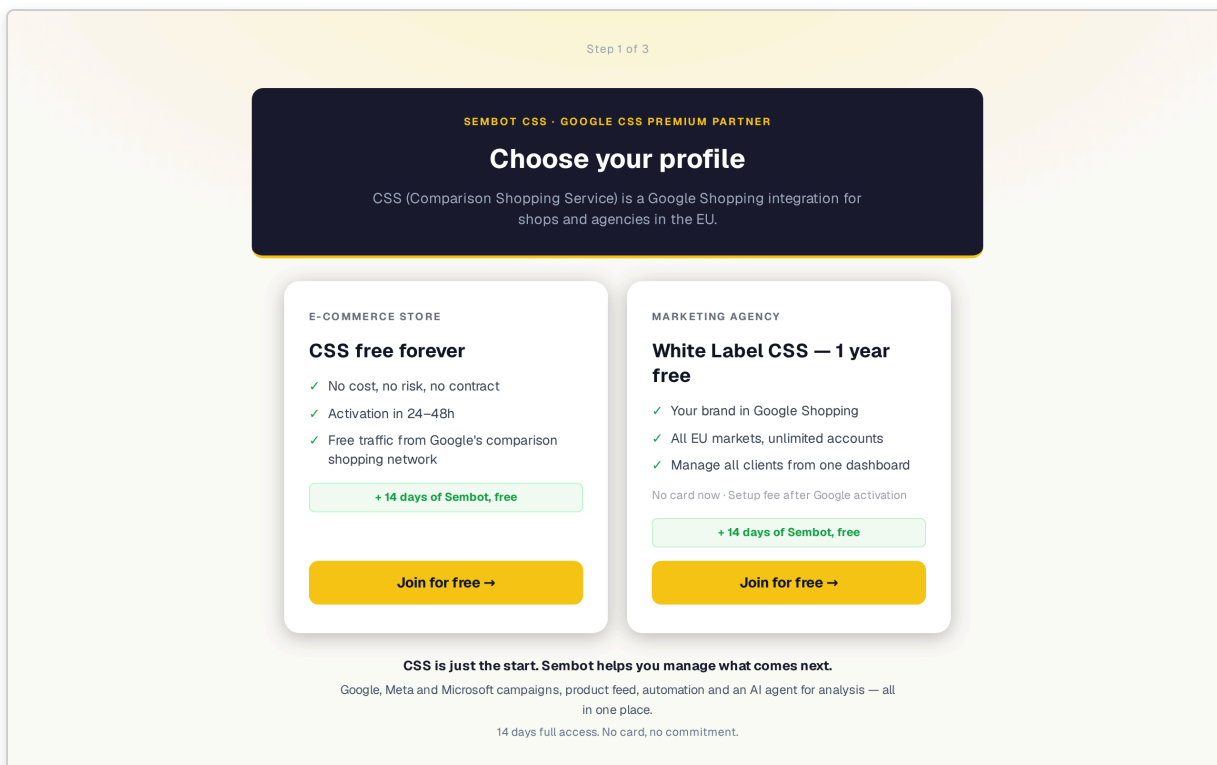
Rozdział B — Ścieżka CSS Agency

auth/css → register?type=css-agency → /css/agency → (mailto) → onboarding/start → onboarding/domain → onboarding/audit-bridge → onboarding/audit

1 Wybór profilu — GET /auth/css (ten sam ekran co Shop krok 1)

Trasa: GET /auth/css

Handler: Auth.ShowCSSPath — internal/handlers/auth.go:108



01-wybor-profilu.png (ekran wspoldzielony z pozostała ścieżka)

CO WIDZI UŻYTKOWNIK

Identyczny ekran jak w ścieżce Shop — użytkownik klika kartę "Marketing Agency" zamiast "E-commerce Store".

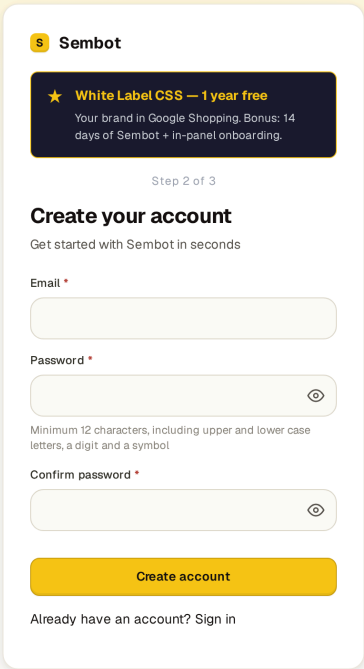
CO SIĘ DZIEJE POD MASKĄ

- CTA karty Agencja prowadzi do /auth/register?type=css-agency (zamiast css-shop) — to jedyna różnica względem ścieżki Shop na tym etapie. Kontekst biznesowy i naprawy — patrz Rozdział A, krok 1.

2 Rejestracja — identyczny formularz, `css_type=css-agency`

Trasa: GET `/auth/register?type=css-agency` → POST `/signup`

Handler: `Auth.ShowSignup` / `Auth.DoSignup` (`signup.go:25, 50`)



The screenshot shows a registration form for Sembot. At the top, there's a banner for 'White Label CSS — 1 year free' with a subtext 'Your brand in Google Shopping. Bonus: 14 days of Sembot + in-panel onboarding.' Below the banner, it says 'Step 2 of 3'. The main heading is 'Create your account' with a subtext 'Get started with Sembot in seconds'. The form has three input fields: 'Email *', 'Password *', and 'Confirm password *'. The Password field has a hint: 'Minimum 12 characters, including upper and lower case letters, a digit and a symbol'. There's a 'Create account' button at the bottom. Below the button, it says 'Already have an account? Sign in'.

03-rejestracja-agency.png

CO WIDZI UŻYTKOWNIK

Ten sam formularz email/hasło co Shop, z bannerem dopasowanym do agencji.

KONTEKST BIZNESOWY

Ten sam formularz co Shop, ale zapisane tu `account_type=css_agency` uruchamia INNY model przychodu (rok gratis + €100/mies. per rynek, zamiast darmo na zawsze) — to biznesowo najważniejsza rzecz zbierana na tym ekranie, mimo że wizualnie nic się nie różni.

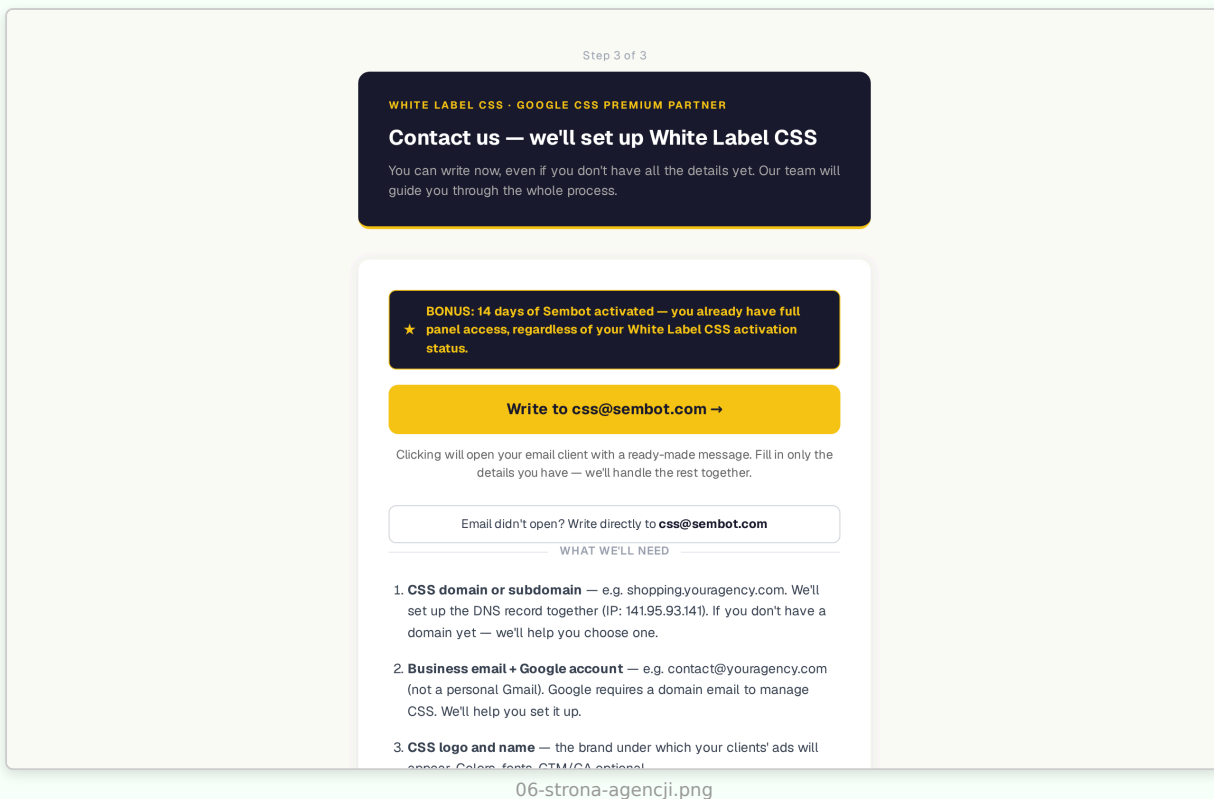
CO SIĘ DZIEJE POD MASKĄ

- Jedyna różnica w `DoSignup` względem ścieżki Shop: destynacja po rejestracji to `/css/agency` zamiast `/css/shop` (`signup.go:111`).
- `account_type = AccountType::CSS_AGENCY` jest zapisywany na modelu `User` — to pole odczytuje później `CssOnboardingController::startTrial()` żeby zdecydować czy wysłać `CssAgencyWelcomeMail`.
- P11:** `?source=` działa identycznie jak w kroku A2 (ten sam handler) — patrz tam po szczegóły.

3 Strona agencji — GET /css/agency (trial znów automatyczny — P8 cofnięte 2026-07-14) **NAPRAWIONE**

Trasa: GET /css/agency

Handler: CSSOnboarding.ShowAgency (css_onboarding.go:194)



CO WIDZI UŻYTKOWNIK

Hero "Contact us — we'll set up White Label CSS". Box: "BONUS: 14 days of Sembot activated — you already have full panel access, regardless of your White Label CSS activation status." Lista wymagań (domena/subdomena CSS, email domenowy + konto Google, logo i nazwa) widoczna zawsze. Duży żółty przycisk "Write to css@sembot.com →" (prefillowany mailto, bez linijki o bonusie — nie jest już potrzebna).

KONTEKST BIZNESOWY

Trzecia iteracja tego samego pytania biznesowego w ciągu trzech tygodni: kiedy trial Sembota ma startować dla Agency, skoro aktywacja White Label przez Google trwa 2-4 tygodnie (dłużej niż sam trial)? Historia pokazuje że to nie jest oczywisty wybór — firma świadomie przetestowała oba podejścia i wróciła do pierwszego, po jawnym zaakceptowaniu ryzyka które ono niesie.

CO SIĘ DZIEJE POD MASKĄ

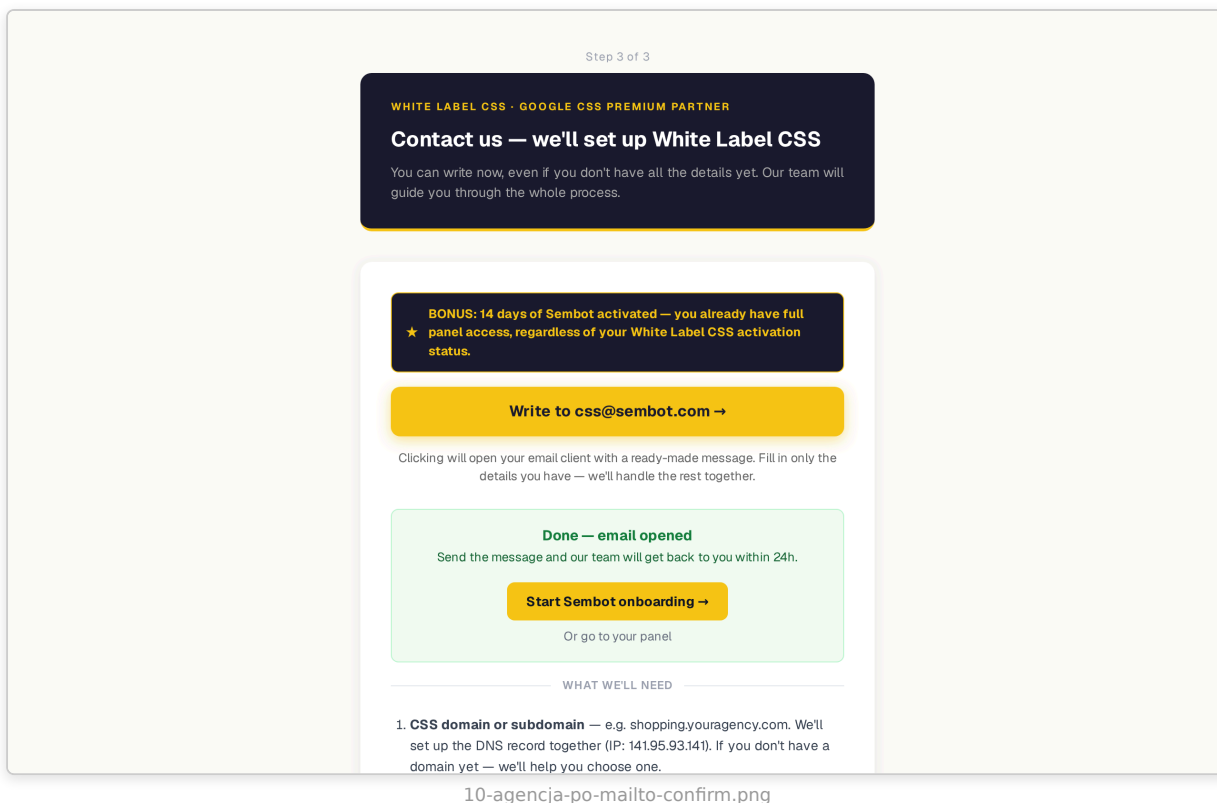
- **Stan pierwotny (od 2026-06-25, commit 6e959f03c9)**: trial nie startował automatycznie dla `account_type=css_agency` — wyjątek wpisany wprost w `UserRepository::createAccount()` (Laravel) z komentarzem o 2-4-tygodniowej aktywacji. `ShowAgency` (proxy_sembot) obchodził to jednak fire-and-forgetowym wywołaniem `CSSStartTrial` przy KAŻDEJ wizycie na stronie (commit b47f2b4) — czyli trial i tak startował automatycznie, tylko przy pierwszym GET zamiast przy rejestracji, i to wielokrotnie (raz na każdą wizytę, nieszkodliwie ale niepotrzebnie).
- **P8 (2026-07-08, commit 42802aa)**: usunięto auto-start z `ShowAgency`. Trial aktywowany ręcznie przez CS, na żądanie usera, przez endpoint superadmina `POST /users/{user}/subscription/trial/extend (whitelist subscription_admin_user_ids)`. Cel: CS aktywuje trial w momencie realnego startu wdrożenia CSS, żeby 14 dni faktycznie się z nim pokrywało.

- **Cofnięcie (2026-07-14):** po rozmowie z przełożonym potwierdzono, że obowiązująca dziś polityka produktu to trial startujący automatycznie przy rejestracji dla WSZYSTKICH kont, bez wyjątków per typ — świadoma decyzja biznesowa, mimo znanego ryzyka (trial może wygasnąć zanim CSS ruszy). Usunięty warunek `$accountType !== AccountType::CSS_AGENCY` w `UserRepository.php:194` — trial i `CssAgencyWelcomeMail` startują teraz w tym samym miejscu co dla każdego innego konta, przy `POST /api/v1/register`.
- **Ważna różnica względem stanu sprzed P8:** to NIE jest zwykłe cofnięcie do starego kodu. Auto-start dziś dzieje się raz, przy rejestracji (Laravel), a nie przy każdej wizycie na `/css/agency` (proxy_sembot) jak przed P8 — czyli finalna wersja unika oryginalnej wady (wielokrotny fire-and-forget) przy zachowaniu prostoty (jeden strzał, deterministyczny moment).
- Mechanizmy z P8 (`CssOnboardingController::startTrial()` , `SubscriptionController::extendTrial()`) zostały w kodzie — nieużywane dziś w normalnym flow dla Agency, ale nieszkodliwe (idempotentne) i przydatne jako ręczne narzędzia CS (np. przedłużenie trialu jeśli White Label się opóźni).
- **Powiązany finding (P10):** w kodzie nie ma dziś egzekwowanego paywalla nigdzie w produkcie. "Aktywacja trialu" nie odblokowuje więc niczego technicznie — to pole informacyjno-raportowe, nie bramka dostępu. Nie unieważnia to decyzji z 2026-07-14 (spójność komunikatu z resztą produktu wciąż się liczy), ale zmienia praktyczne uzasadnienie "dlaczego to ważne".
- Pełna historia decyzji i uzasadnienie: `proxy_sembot/docs/NOTES_CssOnboardingOpenItems.md`.

4 Po kliknięciu mailto — potwierdzenie + zmiana CTA

Trasa: client-side (Alpine.js, brak requestu do serwera)

Handler: `css_onboarding_agency.templ:52-72 — x-data="{emailClicked: false}"`



CO WIDZI UŻYTKOWNIK

Po kliknięciu "Write to css@sembot.com →" (otwiera klienta poczty, `x-on:click="emailClicked=true"`): zielony box "Done — email opened" + nowe CTA "Start Sembot onboarding →" (primary) i "Or go to your panel" (secondary, szary link).

KONTEKST BIZNESOWY

mailto jako jedyna droga zgłoszenia to świadomy kompromis kosztowy MVP (P6 — formularz zamiast mailto — świadomie odłożony poza zakres). Ryzyko biznesowe: serwer nie wie czy user faktycznie wysłał wiadomość, więc część leadów może się zgubić bez żadnego śladu w systemie — CS dowiaduje się o zgłoszeniu tylko jeśli mail faktycznie dotarł.

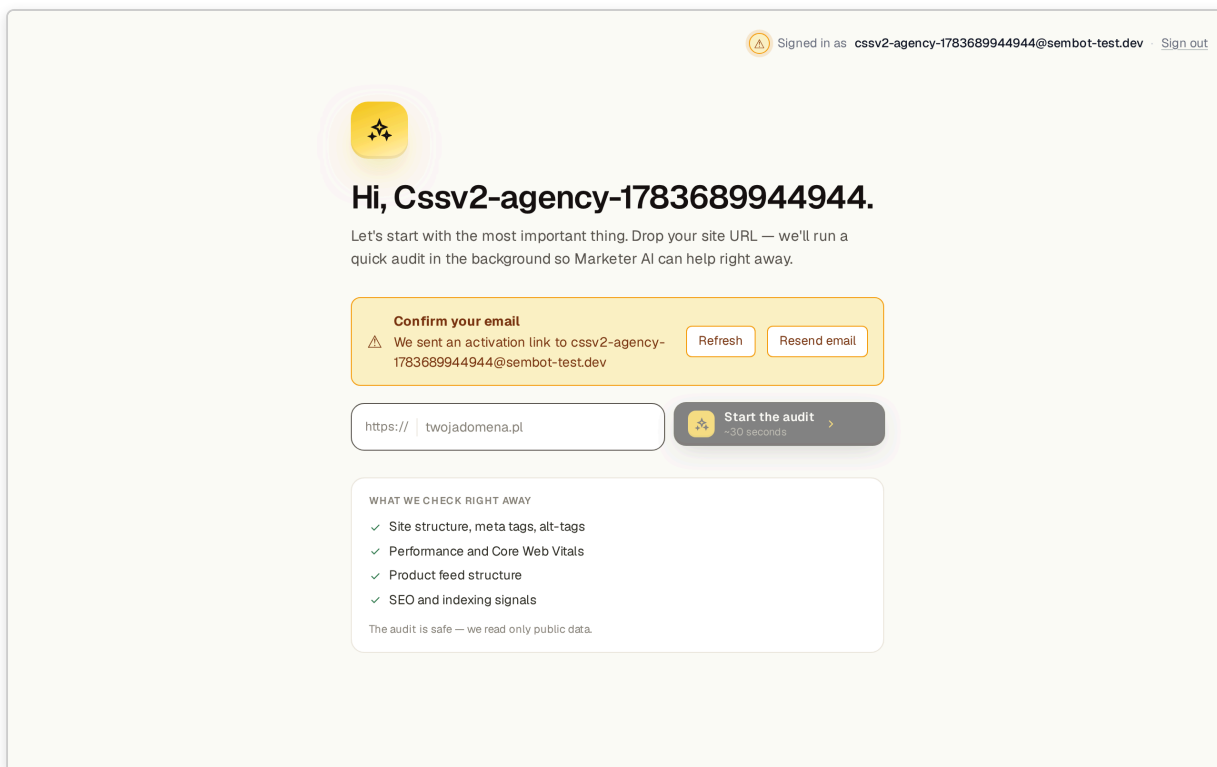
CO SIĘ DZIEJE POD MASKĄ

- Przed P5 (DOC_CssOnboardingFunnel.md) ten sam stan pokazywał "Przejdź do panelu →" prowadzące na `/` (dashboard root, bez scope) — agencja bez projektu łądowała na pustym dashboardie. Po P5 (commit `3e39005`) primary CTA prowadzi na `/onboarding/start`, czyli faktycznie zaprasza do wizarda.
- To jest CZYSTO client-side toggle — mailto: nie generuje żadnego requestu HTTP który serwer mógłby zaobserwować; system nie wie czy user faktycznie wysłał email (opisane jako known-limitation w DOC_CssOnboardingFunnel.md, punkt "mailto jako główna akcja agencji").
- Przycisk "Start Sembot onboarding" nie zależy od statusu trialu (`/onboarding/start` nie sprawdza subskrypcji/trialu — patrz P10) — user może wejść do platformy natychmiast.
- **Zmiana 2026-07-14:** prefillowany mailto (`mailtoCSS()`) miał wcześniej dodatkową liniijkę „PS. Proszę też o aktywację bonusu: 14 dni Sembota" (dodaną przez P8) — usunięta, bo trial znów aktywuje się automatycznie przy rejestracji, CS nie musi być o nic proszony.

5 Wizard Sembota, krok 1 — GET /onboarding/start → /onboarding/domain

Trasa: GET /onboarding/start → 303 → GET /onboarding/domain

Handler: Onboarding.Start (onboarding.go:125) → Onboarding.ShowDomain (onboarding.go:197)



11-onboarding-domain-agency.png

CO WIDZI UŻYTKOWNIK

"Hi, {imię}. Let's start with the most important thing. Drop your site URL." Pole domeny + CTA "Start the audit (~30 seconds)". Widoczny żółty banner "Confirm your email — we sent an activation link to {email} [Refresh] [Resend email]" dopóki email nie jest zweryfikowany.

KONTEKST BIZNESOWY

Agencja, w przeciwieństwie do Shopu, nie ma automatycznie tworzonego projektu przy rejestracji — dopiero wejście do wizarda Sembota to pierwszy realny sygnał zaangażowania w platformę, niezależnie od tego czy/kiedy CSS się aktywuje.

CO SIĘ DZIEJE POD MASKĄ

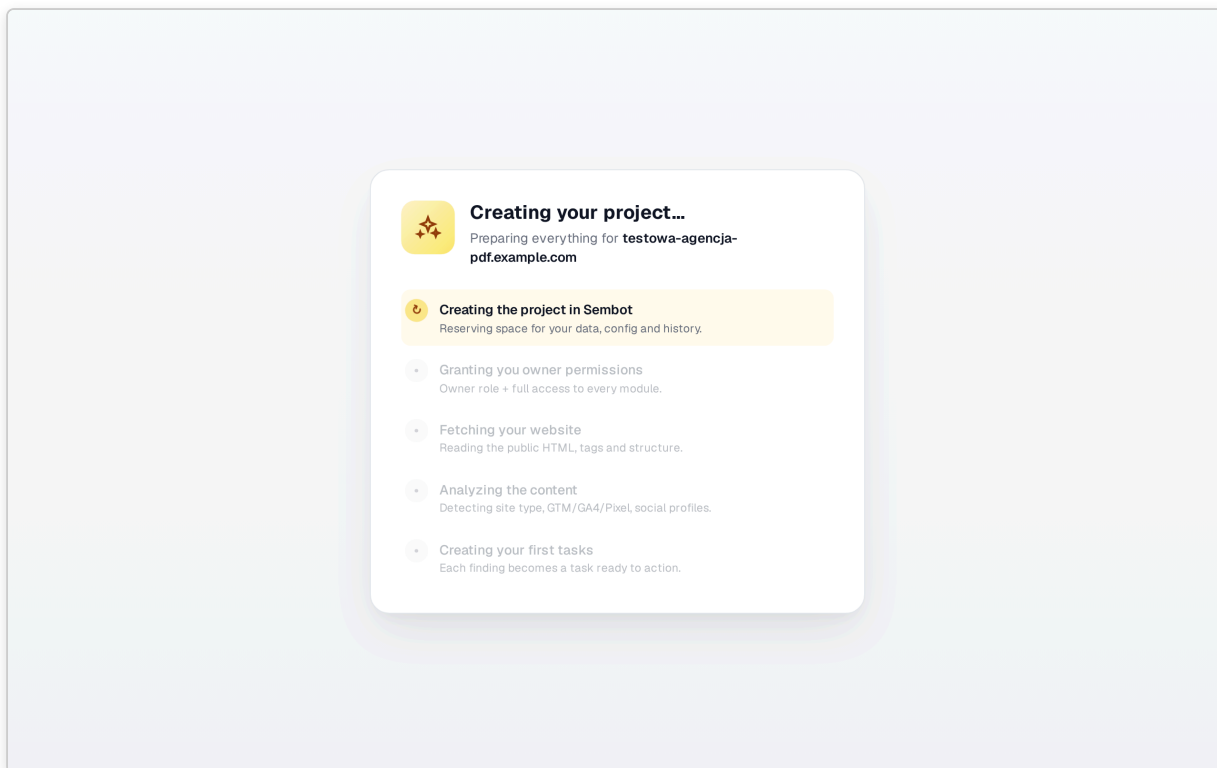
- Start sprawdza `firstProjectDest()` — agencja świeżo zarejestrowana NIE MA projektu (w przeciwieństwie do Shopu, gdzie `CSSCreateProject` tworzy projekt od razu), więc trafia na krok 1 wizarda (`stepDomain`) zamiast na `/tasks` .
- **Realny finding z tej sesji (kod, nie tylko obserwacja):** `SaveDomain` (POST, `onboarding.go:257-282`) blokuje submit dopóki `fetchProfileBits(...).EmailVerified == false` — dla ścieżki `Agency` email NIE jest auto-weryfikowany server-side (w przeciwieństwie do Shopu, gdzie `createShopProject` jawnie woła `markEmailAsVerified()`). Agencja MUSI kliknąć prawdziwy link weryfikacyjny z emaila zanim wizard przepuści submit domeny.
- **Dodatkowy finding:** link weryfikacyjny w mailu (`{new_front_url}/auth/verify/{token}` , generowany przez `App\Notifications\VerifyEmail::verificationUrl()`) wskazuje na trasę której **proxy_sembot w ogóle nie rejestruje** — sprawdzone przez pełny przegląd `cmd/server/main.go` pod kątem `/auth/verify` . Jedyny działający sposób weryfikacji w tej sesji to bezpośrednie wywołanie Laravel API (`GET /api/v1/email/verify/{token}`) z pominięciem frontu. W obecnym stanie realny użytkownik klikający link z prawdziwego maila prawdopodobnie trafia na 404 zamiast zostać zweryfikowanym.

- **Naprawione w sesji 2026-07-07:** dodano przycisk "Resend email" do banneru — wcześniej jedyną akcją było "Refresh" (sprawdź czy już zweryfikowano), bez opcji ponownego wysłania gdy mail nie dotarł. Nowy endpoint `POST /onboarding/resent-verify`.

6 Bridge — tworzenie projektu w tle

Trasa: `POST /onboarding/domain` → 303 → `GET /onboarding/audit-bridge` (polling `GET .../status`)

Handler: `Onboarding.SaveDomain` → `Onboarding.ShowAuditBridge` (`onboarding.go:2808`) / `BridgeStatus` (`onboarding.go:2839`)



12-onboarding-bridge-agency.png

CO WIDZI UŻYTKOWNIK

Checklist z żywym statusem: "Creating the project in Sembot" (aktywny spinner) → "Granting you owner permissions" → "Fetching your website" → "Analyzing the content" → "Creating your first tasks" (kolejne kroki wyszarzone, czekają).

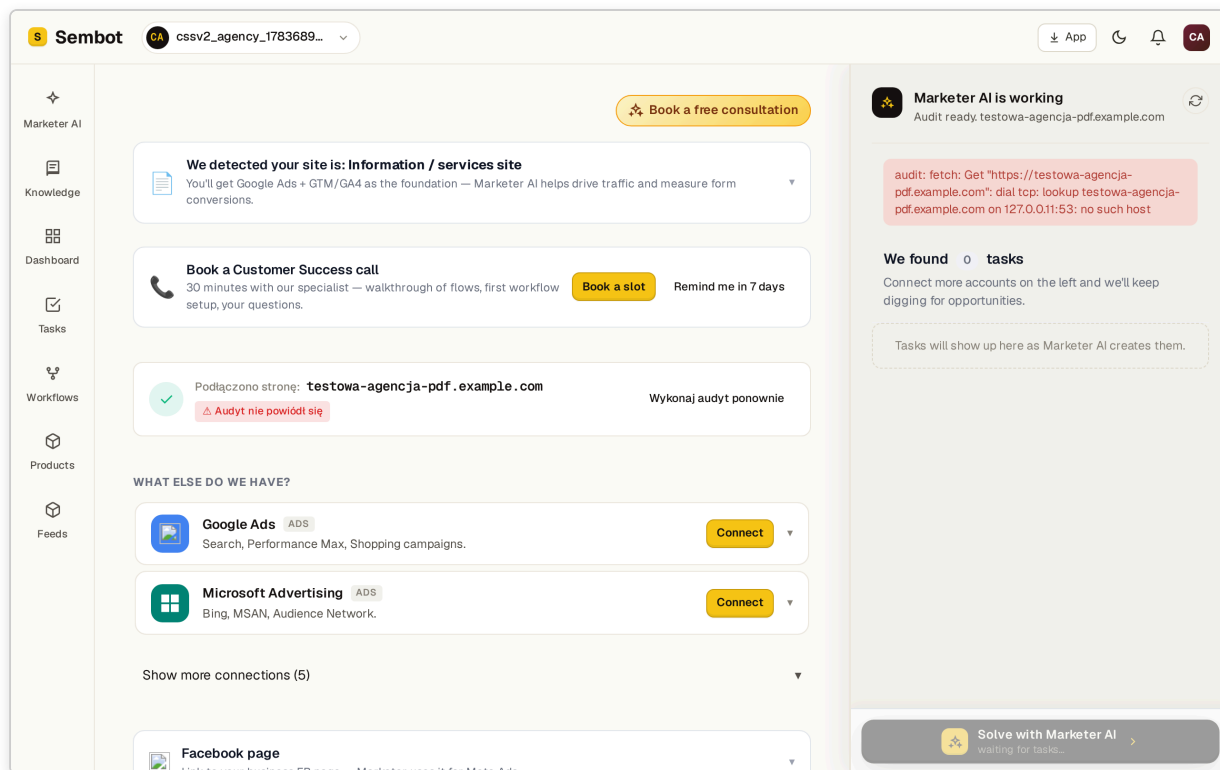
CO SIĘ DZIEJE POD MASKĄ

- `SaveDomain` odpala tworzenie projektu w goroutine (`go func(...)`) i OD RAZU przekierowuje na bridge — projekt tworzy się asynchronicznie żeby user nie czekał ~11s (rola Owner + ~100 uprawnień) na zablokowanym submit.
- Bridge page (`htmx hx-trigger="load delay:500ms, every 1000ms"`) pyta `/onboarding/audit-bridge/status`; handler sprawdza `audit.State.ProjectReady()` (czyli `ProjectSlug != ""`) i gdy gotowe, zwraca header `HX-Redirect` na `/p/{ws}/{proj}/onboarding/audit`.
- Ręczna weryfikacja (bezpośredni `fetch` do `/status` z tej samej sesji) potwierdziła że serwer zwraca poprawny `HX-Redirect` już po ~2-4s od submitu. W tej sesji zaobserwowano także, że automatyczne przekierowanie klienta (`htmx polling`) czasem nie następuje mimo gotowego nagłówka po stronie serwera — do zweryfikowania czy to tylko artefakt środowiska testowego (wiele równoległych rejestracji, obciążony php-fpm) czy realny problem z inicjalizacją `htmx` na tej stronie.

7 Cel osiągnięty — realny panel Sembota (Marketer AI + audyt)

Trasa: GET /p/{ws}/{proj}/onboarding/audit

Handler: Onboarding.ShowAudit (onboarding.go:2008)



13-onboarding-audit-agency.png

CO WIDZI UŻYTKOWNIK

Pełny panel Sembota: sidebar (Marketer AI, Knowledge, Dashboard, Tasks, Workflows, Products, Feeds), lewa kolumna z wykrytym typem strony i listą integracji do podłączenia (Google Ads, Microsoft Advertising, Facebook...), prawy panel "Marketer AI is working" z wynikiem audytu.

KONTEKST BIZNESOWY

Dowód że mechanizm konwersji CSS→Sembot faktycznie działa dla Agencji przy poprawnym przejściu — mimo dwóch realnych przeszkód po drodze (brak trasy weryfikacji email w kroku 5, niepewny auto-redirect w kroku 6), user który je pokona trafia do prawdziwej wartości produktu, tak samo jak Shop.

CO SIĘ DZIEJE POD MASKĄ

- To jest DOKŁADNIE ten sam `OnboardingAudit` widok co dla ścieżki Shop (krok 5/A5) — jeden szablon, jedna logika, niezależnie od tego którą ścieżką (Shop czy Agency) user dotarł do projektu.
- Panel prawy pokazuje realny błąd audytu (`dial tcp: lookup ... no such host`) — bo w tym przebiegu testowym domena była fikcyjna (`final2-agency-pdf.example.com`). Z realną domeną ten sam panel pokazuje właściwe findings i automatycznie tworzy taski (`audit.RunAndPersist` w `ShowAudit` , linia ~2059-2076).
- To potwierdza że — mimo błędu w kroku B5 (brak trasy weryfikacji email) i niepewności w kroku B6 (auto-redirect z bridge) — ścieżka Agency PRZY RĘCZNYM obejściu tych dwóch punktów faktycznie i poprawnie prowadzi usera do prawdziwego, działającego onboardingu Sembota.

Rozdział C — Treści maili

3 maile transakcyjne w całym flow — pełna treść, trigger, odbiorca, kontekst biznesowy

1 CssShopWelcomeMail — powitanie po CSS Shop

Kiedy: `CssOnboardingController::shopNotify()` — TYLKO gdy request zawiera `project_url` (czyli `CSSCreateProject` wcześniej się powiodło)

Do kogo: User (email podany przy rejestracji)

Temat: Twój CSS jest w kolejce aktywacji — {shopName}

Zgłoszenie CSS przyjęte ✓

Projekt **{shopName}** jest gotowy w Sembocie. Nasz zespół aktywuje Twój CSS w ciągu **24-48 godzin**.

[[Przejdź do panelu projektu →](#)]

BONUS: 14 dni Sembota aktywowane

Zanim CSS zostanie aktywowany przez Google, możesz już teraz:

- ✓ Sprawdzić jak Twoje produkty wyglądają w panelu
- ✓ Skonfigurować kampanie Google Shopping
- ✓ Przeglądać i optymalizować produkty
- ✓ Projekt „{shopName}” stworzony w Sembocie
- ✓ Feed produktowy dodany — import zaczyna się w ciągu minut
- ✓ Zgłoszenie wysłane do zespołu CSS
- 🕒 Aktywacja CSS przez Google — nasz zespół skontaktuje się z Tobą w 24-48h

Pytania? Napisz na css@sembot.com — jesteśmy tu, żeby pomóc.

KONTEKST BIZNESOWY

Jedyny mail do usera w całej ścieżce Shop. Buduje poczucie że coś się realnie dzieje podczas 24-48h oczekiwania na aktywację CSS przez Google, i przypomina o bonusie Sembota (re-engagement gdyby user zamknął panel od razu po rejestracji). "Aktywowane" jest tu prawdziwe — w przeciwieństwie do wcześniejszej wersji maila agencyjnego (patrz e3), bo trial Shop faktycznie startuje natychmiast i mieści się w oknie 24-48h.

2

CssShopNotifyMail — zgłoszenie do zespołu CS

Kiedy: `CssOnboardingController::shopNotify()` — ZAWSZE przy submisie formularza sklepu, niezależnie czy projekt się utworzył

Do kogo: `css@sembot.com` (zespół CS — NIE user)

Temat: CSS Shop Onboarding — {shopName} / GMC: {gmclId}

Nowe zgłoszenie CSS Shop Onboarding

Nazwa sklepu: {shopName}

GMC ID: {gmclId}

URL feeda: {feedUrl}

Domena: {domain}

Email użytkownika: {user.email}

Projekt w Sembocie: {projectUrl}

KONTEKST BIZNESOWY

To jest realny trigger pracy zespołu CS — od tego maila zaczyna się 24-48h licznik aktywacji CSS obiecany userowi na stronie sukcesu (krok A4). Wysyłka to zwykły synchroniczny `Mail::send` bez retry/queue — jeśli ten mail zaginie (spam, błąd SMTP), CS nigdy się nie dowie o zgłoszeniu, a user będzie czekał na aktywację która nigdy nie nadejdzie.

3 CssAgencyWelcomeMail — powitanie agencji + bonus do odebrania

Kiedy: `CssOnboardingController::startTrial()` — gdy `$isFirstTrial && account_type=css_agency`. **Po P8 (2026-07-08): wywoływane ręcznie przez CS** (endpoint `superadmina trial/extend`), NIE automatycznie przy rejestracji ani przy wizycie na stronie.

Do kogo: User (email podany przy rejestracji)

Temat: Twój White Label CSS — co przygotować i co dalej

Rejestracja agencji przyjęta ✓

Nasz zespół skontaktuje się z Tobą, żeby uruchomić **White Label CSS**. Aktywacja przez Google zajmuje zazwyczaj **2-4 tygodnie** od przesłania kompletnych danych.

CO PRZYGOTOWAĆ PRZED KONTAKTEM Z ZESPOŁEM

1. Domena lub subdomena CSS
2. Email domenowy (nie prywatny Gmail)
3. Konto Google (powiązane z emailen domenowym)
4. Logo i nazwa CSS

[Napisz do nas: css@sembot.com →]

Bonus do odebrania: 14 dni Sembota

Ten bonus nie aktywuje się automatycznie — zależy nam, żeby te 14 dni realnie pokrywało się z uruchomieniem Twojego White Label CSS, a nie wygasło zanim wdrożenie się zacznie (trwa ono 2-4 tygodnie).

Aktywuj go w dowolnym momencie, pisząc na css@sembot.com — najlepiej gdy ruszamy z pracą nad Twoim CSS.

Gdy będzie aktywny, zyskasz dostęp do:

- ✓ Pełnej platformy Sembot — kampanie Google, Meta i Microsoft
- ✓ Konfiguracji projektu dla jednego z klientów agencji
- ✓ Feedu produktowego i automatyzacji

[Zobacz platformę Sembot →]

Pytania? Napisz na css@sembot.com — jesteśmy tu, żeby pomóc.

KONTEKST BIZNESOWY

Ten mail przeszedł największą zmianę w sesji (P8, 2026-07-08 i 2026-07-09): sekcja bonusu mówiła wcześniej "aktywowane" (czas przeszły, nieprawdziwe — trial startował automatycznie w dniu rejestracji, długo przed realną aktywacją CSS). Po P8 mail idzie dopiero gdy CS ręcznie aktywuje trial, więc w praktyce jest to też nieformalne potwierdzenie dla usera że ktoś z zespołu faktycznie zareagował na jego zgłoszenie — nie tylko automat przy rejestracji.

Tabela plików kluczowych

Plik	Rola
proxy_sembot/internal/handlers/auth.go	ShowCSSPath (/auth/css), ShowSignup/DoSignup (rejestracja + routing po css_type)
proxy_sembot/internal/handlers/css_onboarding.go	ShowShop, SubmitShop, ShowShopDone, ShowAgency
proxy_sembot/internal/handlers/onboarding.go	Start, ShowDomain, SaveDomain, ShowAuditBridge, BridgeStatus, ShowAudit, ResendVerification — wizard Sembota
proxy_sembot/internal/middleware/onboarding_redirect.go	Middleware wpychający nieonboardowanych userów do wizarda
proxy_sembot/internal/middleware/require_project.go	Guard projektu dla /p/{ws}/{proj}/... gdy proj=0
proxy_sembot/internal/views/pages/css_path.templ	Wybór profilu Shop/Agency
proxy_sembot/internal/views/pages/css_onboarding_shop.templ	Formularz sklepu (krok 3 Shop)
proxy_sembot/internal/views/pages/css_onboarding_shop_success.templ	Strona sukcesu Shop
proxy_sembot/internal/views/pages/css_onboarding_agency.templ	Ekran agencji + mailto CTA (Alpine emailClicked)
proxy_sembot/internal/views/pages/onboarding_domain.templ	Wizard krok 1 (Agency) — pole domeny, banner weryfikacji email + resend
proxy_sembot/internal/api/css.go	CSSShopNotify, CSSStartTrial (klient Go → Laravel)
proxy_sembot/internal/api/projects.go	CreateProject, CSSCreateProject (POST /css/shop/project)
sembot-laravel/app/Http/Controllers/Api/Css/CssOnboardingController.php	shopNotify, createShopProject, startTrial
sembot-laravel/app/Http/Controllers/Auth/OAuthController.php	Social login/signup (P9 — dziś nie przenosi css_type, patrz Status MVP)
sembot-laravel/app/Http/Middleware/HasSubscription.php , Paywall.php	Middleware trialu/paywalla — dziś niepodpięte pod żaden route (P10)
sembot-laravel/app/Jobs/Projects/StoreProject.php	Tworzenie projektu — tenant assignment (linia 89, wymaga ≥1 tenanta w landlord DB)
sembot-laravel/app/Notifications/VerifyEmail.php	Generuje link weryfikacyjny {front_url}/auth/verify/{token} — trasa BRAKUJE w proxy_sembot

sembot-laravel/app/Mail/CssShopWelcomeMail.php , CssShopNotifyMail.php , CssAgencyWelcomeMail.php	3 maile transakcyjne — pełna treść w Rozdziale C
locales/pl/css.json , locales/en/css.json	Copy całego flow CSS (UI, nie maile)
proxy_sembot/internal/handlers/signup.go	ShowSignup/DoSignup — czytają middleware.Source(r) , przenoszą przez ukryte pole formularza (P11)
proxy_sembot/internal/middleware/source.go	Nowy (2026-07-13). WithSource — first-touch capture ?source= w cookie, analogicznie do WithLocale ; SanitizeSource (przeniesione z signup.go)
proxy_sembot/internal/middleware/auth.go	ctxSource , SetSource / Source — context accessors (2026-07-13)
proxy_sembot/cmd/server/main.go	middleware.WithSource() wpięty globalnie w chain, zaraz po WithLocale (2026-07-13)
sembot-laravel/app/Listeners/Users/SendRegisterAnalyticsToPostHog.php	Event PostHog user_registered — wzbogacony o language/account_type/source (P11)
sembot-laravel/app/Services/PostHog/PostHogService.php	track() — nowy param \$userIdentifyOnceData (\$set_once, P11)
sembot-laravel/app/Repositories/UserRepository.php	Zmieniony (2026-07-14). createAccount() — usunięty wyjątek dla CSS_AGENCY , trial + CssAgencyWelcomeMail startują przy rejestracji dla wszystkich kont (cofnięcie P8)
sembot-laravel/resources/views/emails/css-agency-welcome.blade.php	Zmieniony (2026-07-14). Sekcja BONUS z powrotem na "aktywowane", CTA z powrotem na "Zacznij onboarding w Sembocie"
sembot-laravel/tests/Feature/Repositories/UserRepositoryTest.php	Nowy (2026-07-14). Auto-trial + welcome mail przy rejestracji dla CSS Agency/Shop/regular
proxy_sembot/internal/middleware/i18n.go	Zmieniony (2026-07-14). WithLocale — deterministyczny fallback dla niewspieranego ?lang= zamiast Accept-Language sniffingu
proxy_sembot/internal/middleware/i18n_test.go	Nowy (2026-07-14). 4 przypadki dla WithLocale , w tym regres na niedeterministyczny sniffing
proxy_sembot/docs/NOTES_CssOnboardingOpenItems.md	Nowy (2026-07-14). Robocza notatka — historia i uzasadnienie decyzji o cofnięciu P8, rekomendacje z przeglądu UX/analitika/admin/marketing

Status MVP i otwarte pytania

✅ **Zaimplementowane i zweryfikowane działające:** P1 (CTA na success page Shop), P2 (email do usera po CSS Shop), P3 (blok Sembota na /auth/css), P4 (email welcome agencji), P5 (zmiana CTA agencji po mailto), P11 (atrybucja `source` + wzbogacony event PostHog `user_registered`), **P11-fix (2026-07-13)** — propagacja `source` przez `/auth/css` (`middleware.WithSource`) + korekta domeny (`new.sembot.com`) + wdrożenie i weryfikacja linków na `css.sembot.com` (17 linków, 8 plików, zero linków do `app.sembot.com`), **i18n-fix (2026-07-14)** — deterministyczny fallback dla niewspieranego `?lang=` (`middleware.WithLocale`) — patrz sekcja "Analityka i atrybucja".

↩️ **Zaimplementowane, potem cofnięte na żądanie biznesu:** P8 (trial agencji na żądanie zamiast auto-startu) — wdrożone 2026-07-08, świadomie cofnięte 2026-07-14 po decyzji biznesowej o jednolitej polityce trialu dla wszystkich kont. Pełna historia: sekcja P8 wyżej i "Oś czasu".

🚫 **Świadomie poza zakresem MVP:**

- **P6** — formularz zamiast mailto dla agencji (ryzyko zgubionych leadów, patrz krok B4).
- **P7** — interaktywna edukacja platformy przy polu feed URL (kosmetyczny UX).
- **P9** — rejestracja przez Google/Facebook/LinkedIn. `/login` ma social buttons, `/signup` nigdy ich nie miał (preexisting gap całego produktu, nie decyzja CSS onboarding). Plan rozpisany (4 kroki, proxy_sembot + sembot-laravel) w `DOC_CssOnboardingFunnel.md` — nie do zweryfikowania w tym środowisku (puste FB/LinkedIn credentials, Google redirect URI nie pasuje do dynamicznego URL-a devcontainera).
- Ochrona przed duplikatami projektu w `/css/shop`.

⚠️ **P10 (finding informacyjny, nie zadanie):** w kodzie nie ma dziś egzekwowanego paywalla nigdzie w produkcie — patrz sekcja "Oś czasu" i krok B3. Nie dotyczy tylko CSS.

⚠️ **Znaleziska z tej sesji (świadomie odłożone — decyzja biznesowa/produktowa, nie techniczna):**

- **[Odłożone do sesji mergowej]** Trasa `/auth/verify/{token}` (link weryfikacyjny w mailu rejestracyjnym) nie jest zarejestrowana w proxy_sembot — dotyczy WSZYSTKICH rejestracji przez nowy front, nie tylko CSS Agency, ale to właśnie ścieżka Agency jest tą, w której weryfikacja emaila faktycznie blokuje dalszy postęp (SaveDomain). Decyzja: to szerszy temat niż CSS onboarding, do ogarnięcia jak dev usiadzie do mergowania całości.
- **[Niepotwierdzone]** Auto-redirect z bridge page (htmx polling) czasem nie triggeruje mimo gotowego `HX-Redirect` po stronie serwera — może być artefakt środowiska testowego z wieloma równoległymi rejestracjami, nie zweryfikowano w izolacji.

Pytania biznesowe nierozstrzygnięte: finalna akceptacja copy emaili (Rozdział C), akceptacja ryzyka mailto jako głównej akcji agencji, aktualność czasów aktywacji CSS (24-48h shop / 2-4 tygodnie agencja), czy maile transakcyjne powinny mieć wariant EN, czy CS potrzebuje automatycznego potwierdzenia zwrotnego dla usera po ręcznej aktywacji trialu, kiedy `css.sembot.com` uruchomi kolejne wersje językowe UE poza dzisiejszymi PL/EN (wzorzec URL z `source=css-<kod>` już gotowy i udokumentowany w sekcji "Analityka i atrybucja" — czeka tylko na faktyczną ekspansję językową strony, nie na kod).